

Website themes

A theme controls how your website looks: layout, navigation, colors, typography, and output-specific styling. Every *Calepin* site uses one. The default theme works with zero configuration, and when you want more control, you can copy any built-in theme into your project and edit it like the rest of your source files.

Choosing

Set theme in your site's `calepin.toml`:

```
theme = "calepin"           # the default documentation theme
theme = "academic"          # a built-in academic site theme
theme = "themes/my-theme"    # a local theme directory
theme = false                # no theme: raw, unstyled output
```

The same values work with `--theme` on `calepin compile` and `calepin watch`, and inside a document with `#calepin.setup(theme: ...)`. When several are set, the command line wins, then the document, then `calepin.toml`.

Built-in

Calepin ships with two built-in themes:

- **calepin**: the default documentation site layout, with sidebar navigation, a top bar, previous and next page links, a table of contents, dark mode, copy buttons on code blocks, and a view switcher for rendered HTML, Typst source, and PDF.
- **academic**: a personal homepage layout with top navigation instead of a sidebar, designed for profile pages, teaching materials, publication lists, projects, talks, and posts. Run `calepin new academic` to scaffold a complete starter site built on it.

Local directory

A local theme is a directory in your project. Point theme at that directory:

```
theme = "themes/my-theme"
```

Use a local directory when you want project-specific HTML, CSS, JavaScript, or paged Typst styling. Missing files fall back to the built-in *calepin* theme, so a small local theme can override just one file.

Customizing

Built-in themes are compiled into the *Calepin* binary, so you cannot edit them directly. Instead, copy one into your project and edit the copy:

```
calepin new theme           # copies the default theme to themes/calepin/
calepin new theme --theme academic # copies the academic theme to themes/academic/
```

Then point your site at the copy:

```
theme = "themes/calepin"
```

The copy is yours: edit its HTML, CSS, JavaScript, and Typst files freely, and check them into version control. *Calepin* upgrades will never touch them.

Keep only the files you actually change. Anything you delete from your copy falls back to the built-in default, so a theme directory containing a single CSS file is valid and makes the customization obvious. Never edit files under `.calepin/`; that directory is regenerated on every build.

Structure and files

A theme can provide three entry files, one per kind of output:

```
themes/my-theme/
  site.html      # layout for website pages
  document.html  # layout for a single document rendered to HTML
  paged.typ      # Typst styling for PDF, SVG, and PNG output
  partials/      # reusable template fragments
  styles/        # CSS files, loaded in filename order
  scripts/       # JavaScript files, loaded in filename order
```

Every file is optional. When an entry file is missing, that kind of output uses the default `calepin` theme instead, including the default's own partials, styles, and scripts. The built-in `academic` theme works this way: it only provides `site.html`, and falls back to the default for single-document HTML and paged output.

One special case: an empty `paged.typ` turns paged styling off entirely, while a missing `paged.typ` inherits the default styling.

`site.html` and `document.html` use the MiniJinja template language, which follows familiar Jinja2 syntax. On website builds, templates receive a `site` object describing the site and current page:

- `site.title`, `site.description`, `site.base_url`: site identity from `calepin.toml`
- `site.logo`, `site.logo_alt`, `site.home_url`, `site.favicon`: branding and site assets
- `site.sidebar`, `site.sidebar_sections`: sidebar navigation entries
- `site.navbar_left`, `site.navbar_center`, `site.navbar_right`: navbar entries by region
- `site.toc`: table of contents for the current page
- `site.page_title`, `site.current_url`: current page metadata
- `site.language`, `site.languages`, `site.translations`: multilingual site data

For the variables available when rendering a single document, see [HTML themes](#).

Here is a small but complete `site.html`:

```
{{ doc.head }}
  {% if site.page_title or site.title %}
    <title>{% if site.page_title %}{{ site.page_title }}{% if site.title %} |
  {{ site.title }}{% endif %}{% else %}{{ site.title }}{% endif %}</title>
  {% endif %}
  {% if site.description %}
    <meta name="description" content="{{ site.description }}">
  {% endif %}
  {% if site.favicon %}
    <link rel="icon" href="{{ site.favicon }}">
  {% endif %}
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/
pico.min.css">
  {% for style in styles %}
    <style>
  {{ style.css }}
    </style>
```

```

    {% endfor %}
{{ doc.body_open }}
{% include "partials/site-header.html" %}
<div class="site-shell">
    {% include "partials/sidebar.html" %}
    <main class="site-main">
        {{ doc.body }}
    </main>
    {% if site.toc %}
    <aside class="site-toc">
        <strong>On this page</strong>
        <ul>
            {% for item in site.toc %}
            <li class="level-{{ item.level }}"><a
href="{{ item.href }}">{{ item.label }}</a></li>
            {% endfor %}
        </ul>
    </aside>
    {% endif %}
</div>
{% for script in scripts %}
<script>
{{ script.content }}
</script>
{% endfor %}
{{ doc.body_close }}

```

And a minimal document.html for single-file HTML output:

```

{{ doc.head }}
<title>{{ doc.title }}</title>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/
pico.min.css">
{% for style in styles %}
<style>
{{ style.css }}
</style>
{% endfor %}
{{ doc.body_open }}
<header class="document-header">
    <a href="index.html">Home</a>
    <button type="button" data-calepin-theme-toggle>Theme</button>
</header>
<main class="container">
    {{ doc.body }}
</main>
{% for script in scripts %}
<script>
{{ script.content }}
</script>
{% endfor %}
{{ doc.body_close }}

```

doc.head, doc.body_open, and doc.body_close come from Typst's generated HTML. Keep them in the template unless you are deliberately replacing the whole document shell.

Partials

A partial is a reusable MiniJinja template fragment stored under `partials/`. Use partials for repeated HTML such as a header, footer, navigation list, search box, or analytics snippet.

Include a partial from `site.html`, `document.html`, or another partial:

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them, so a partial included by `site.html` can read `site.title`, `site.navbar_left`, `styles`, `scripts`, and the other website template variables.

For example, `partials/site-header.html` can render the brand and top navigation:

```
<header class="site-header">
  <nav>
    <ul>
      <li>
        <a href="{{ site.home_url }}">
          {% if site.logo %}
            
          {% elif site.title %}
            {{ site.title }}
          {% else %}
            Home
          {% endif %}
        </a>
      </li>
      {% for item in site.navbar_left %}
        {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
    <ul>
      {% for item in site.navbar_right %}
        {% include "partials/nav-item.html" %}
      {% endfor %}
    </ul>
  </nav>
</header>
```

Then `partials/nav-item.html` can handle links and widgets in one place:

```
{% if item.widget == "theme" %}
<li>
  <button type="button" aria-label="{{ item.label }}" data-calepin-theme-toggle>
    {{ item.label_html }}
  </button>
</li>
{% elif item.widget == "language" and site.languages | length > 1 %}
<li>
  <select aria-label="{{ item.label }}" data-calepin-language-picker>
    {% for language in site.languages %}
      <option value="{{ language.href }}" data-calepin-language-
code="{{ language.code }}" {% if language.active %} selected{% endif %}
>{{ language.label }}</option>
    {% endfor %}
  </select>
</li>
```

```

    </select>
</li>
{% else %}
<li>
    <a href="{{ item.href }}" aria-label="{{ item.label }}" {% if item.active %} aria-
current="page" {% endif %}>
        {{ item.label_html }}
    </a>
</li>
{% endif %}

```

partials/sidebar.html can use section state to render foldable navigation:

```

{% if site.sidebar_sections %}
<aside class="site-sidebar">
    {% for section in site.sidebar_sections %}
    {% if section.items %}
    <details{% if section.active %} open{% endif %}>
        <summary>{{ section.title }}</summary>
        <ul>
            {% for item in section.items %}
            <li><a href="{{ item.href }}" {% if item.active %} aria-current="page" {% endif
%}>{{ item.label_html }}</a></li>
            {% endfor %}
        </ul>
    </details>
    {% endif %}
    {% endfor %}
</aside>
{% endif %}

```

Snippets

When you run `calepin new theme`, *Calepin* writes the shared CSS and JavaScript as normal files in your theme:

```

themes/calepin/
  styles/00-theme.css
  styles/01-code.css
  styles/02-widgets.css
  scripts/00-theme-toggle.js
  scripts/01-language-picker.js
  scripts/02-copy-code.js

```

These files are the pieces that the built-in themes use for typography, syntax highlighting, code output, dark mode, language selection, and copy buttons. They are copied into your theme so you can inspect, edit, delete, or replace them directly.

The numeric prefixes matter only for load order. Theme CSS and JavaScript are loaded in filename order. Put broad variables and base rules first, then component rules, then project-specific overrides:

```

styles/
  00-theme.css
  01-code.css
  02-widgets.css
  90-overrides.css

```

The older snippets template object is still available for compatibility with existing custom themes, but new ejected themes do not need it. Prefer editing the files in `styles/` and `scripts/` because they are explicit and version-controlled with your theme.

Widgets

Each entry in `site.sidebar`, `site.sidebar_sections`, and the navbar regions exposes `href`, `label`, `label_html`, `active`, and `widget`. Plain links have `widget = none`.

Widget entries carry whatever string was configured in `calepin.toml`:

```
[[navbar.item]]
position = "right"
widget = "theme"

[[navbar.item]]
position = "right"
widget = "language"
```

The built-in themes recognize `widget = "theme"` for the dark mode toggle and `widget = "language"` for the language picker. A custom theme can invent its own widget names, such as `search`, and render them by checking `item.widget`.

The bundled widget snippets expect these attributes:

```
<button data-calepin-theme-toggle>Theme</button>
<select data-calepin-language-picker></select>
```

Conditional output

`paged.typ` is a plain Typst file, not a template. When rendering PDF, SVG, or PNG output, *Calepin* inserts it ahead of each page's source, after its own code chunk rules, so your `set` and `show` rules apply to the whole document.

Calepin also stages reusable Typst snippets under `/.calepin/snippets/typst/`. The default theme's `paged.typ` imports `code-block.typ` from there to style source blocks. Use `sys.inputs.at("calepin-target", default: "paged")` when a rule should behave differently for HTML and paged output:

```
#import "/.calepin/snippets/typst/code-block.typ": code-block

#show raw.where(block: true): it => {
  if sys.inputs.at("calepin-target", default: "paged") == "html" {
    it
  } else {
    code-block(it)
  }
}
```

The default theme's view switcher shows each website page as rendered HTML, Typst source, or PDF. The source view reads a JSON script embedded in each generated HTML page, so it always works. The PDF view links to the matching `.pdf` file, which only exists when PDF rendering is enabled. If you build with `--format html`, no PDF files are generated, and the PDF view has nothing to show.